Real-time Interactivity in Hybrid Applications With Web Sockets

Article *in* International Research Journal of Modernization in Engineering Technology and Science · January 2024

DOI: 10.56726/IRJMETS48494

1 author:

Amol Gote
Innova Solutions
8 PUBLICATIONS **3** CITATIONS

SEE PROFILE

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:06/Issue:01/Jauuary-2024 Impact Factor- 7.868 www.irjmets.com

REAL-TIME INTERACTIVITY IN HYBRID APPLICATIONS WITH WEB SOCKETS

Amol Gote[*1]

[*1]Solutions Architect, FinTech, Plainsboro, New Jersey, USA.

DOI : https://www.doi.org/10.56726/IRJMETS48494

ABSTRACT

Web sockets have become a powerful technology for enabling real-time communication and interactivity in modern web applications. This research paper explores the innovative utilization of web sockets to enhance interactivity in hybrid applications. We focus on three distinct use cases: interactive sessions between two web applications, between a web application and a mobile device, and between a web application and a tablet device, all involving two parties. Our study delves deep into each of these use cases, providing comprehensive insights into the tools and frameworks employed to facilitate seamless interactivity. For interactive sessions between two web applications, we detail the specific tools utilized to establish robust communication channels. When it comes to interactions between web applications and tablet/mobile devices, we present two distinct approaches for native mobile apps to receive interactive notifications, enabling real-time data synchronization. Reduced latency, seamless data exchange, and responsive communication are pivotal in creating immersive and engaging experiences. In conclusion, this paper offers a comprehensive exploration of leveraging web sockets for interactivity in hybrid applications across a diverse range of scenarios. Developers and researchers will find valuable insights into the tools, frameworks, and strategies necessary to enhance user engagement and create dynamic two-party interactive applications.

Keywords: Web Sockets, Native Mobile Application, Tablet, Frameworks, Interactivity.

I. INTRODUCTION

In an era of Web 2.0 and the proliferation of diverse computing devices, the demand for interactive user experiences has never been more pronounced. Thus, driving the need for innovative solutions that provide interactive experience between web applications, mobile devices, and tablets. Web sockets have emerged as a potent tool in this endeavor.

Web sockets enable bidirectional, real-time communication between a client (such as a web browser/native mobile app) and a server. Unlike the traditional request/response model, where the client sends a request to the server, waits for a response, and then renders the updated content, web sockets establish a persistent connection that allows both the client and server to exchange data instantly. The traditional request/response model is inherently stateless, in contrast, web sockets offer a stateful, persistent connection that remains open as long as needed, facilitating real-time data exchange. This difference allows for instant updates, reduced latency, and true interactivity.

This research paper explores the potential of web sockets in the realm of hybrid applications. Focusing on three distinct use cases: interactive sessions between two web applications, between a web application and a mobile device, and between a web application and a tablet device—each involving two parties engaged in real-time communication. For each of the use cases, the paper articulates the tools and frameworks necessary to build interactivity between hybrid applications. The paper will also provide a design approach for scaling the Web Sockets based applications.

II. USE CASES FOR WEB SOCKETS IN HYBRID APPLICATIONS

In this section, we will explore three distinct use cases:

- **Interactive Sessions Between Two Web Applications:** This use case involves enabling real-time, interactive communication between two web applications.
- **Interaction Between a Web Application and a Mobile Device:** Here, we delve into scenarios where web sockets facilitate instant communication between a web application and a mobile device. This section also covers alternate ways in which native mobile apps could receive updates through silent push notifications.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:06/Issue:01/Jauuary-2024 Impact Factor- 7.868 www.irjmets.com

- **Interaction Between a Web Application and a Tablet Device:** This use case focuses on using web sockets to enable interaction between web applications and tablet devices.

In all three scenarios, we will consider a practical example of a person taking a consumer loan at a home improvement retail store. In this case, there are two parties involved first the consumer who is taking the loan, and second the retail store representative. The consumer would be opening a link that a retail store representative has sent and apply for the loan using that link, in this case consumer will enter personally identifiable information (PII) and proceed to see loan offers. When the consumer is applying the retore store representative would see that the user is entering information, but PII information would not be visible. Once the applicant has applied then the interactive journey of both parties would start. Both parties would see loan offers on their respective screens and the store representative could drive the conversation based on the offers. The store representative could provide a discount and push a new set of offers, then the applicant would see immediately a new set of offers, both parties can negotiate a rate and then the applicant can proceed in selecting the offer and completing the remaining steps for loan application.

Interactive Sessions Between Two Web Applications

Interactive sessions between two web applications refer to scenarios where web-based applications establish real-time communication channels to enable interactive exchanges between them. This use case is relevant in various contexts, including collaborative document editing, online games, and data sharing. In such scenarios, both web applications establish a connection with a single server, and communication happens seamlessly between the two connected clients.

However, in the case of load-balanced environments, where multiple servers distribute incoming traffic, two client applications could establish connections with different servers. In this situation, a backplane becomes essential. A backplane functions as a publisher-subscriber system, where a client subscribes to a particular topic, and any messages published to that topic are then relayed to the subscribing client.

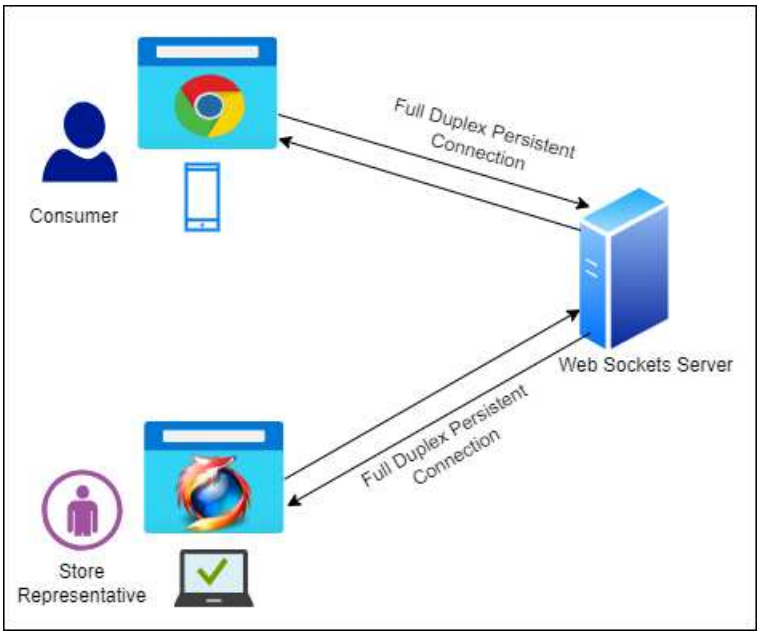

Figure. 1. Single Server

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:06/Issue:01/Jauuary-2024 Impact Factor- 7.868 www.irjmets.com

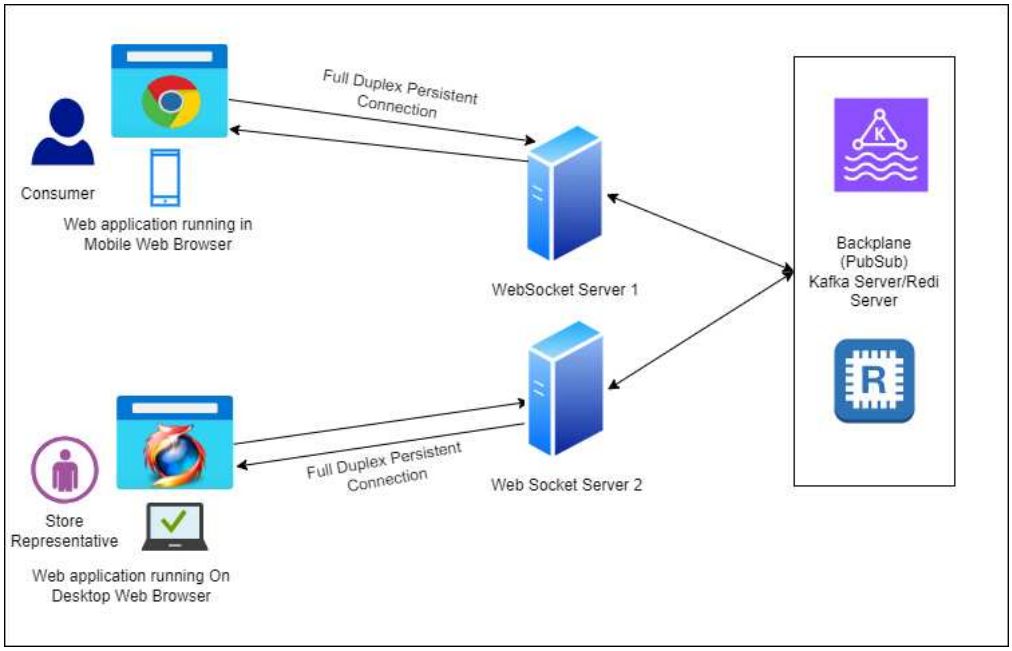

Figure. 2. Load Balanced (Multiple Web Socket Servers)

For implementing interactive sessions between web applications, various frameworks and tools are available, tailored to different technology stacks. Here are some commonly used options:

- Node.js Applications: Socket.io, along with the client JavaScript library "socket.io.js," is a popular choice for Node.js applications.
- Java Spring Boot Applications: The "spring-websocket" maven package, combined with the client JavaScript libraries "sockjs" and "stompJS," is commonly employed for Java Spring Boot applications.
- Backplane Implementation: To build the backplane, you can choose from various publisher-subscriber systems. Redis server is a widely adopted solution, while Kafka can also be leveraged as a robust backplane.

Interaction Between a Web Application and a Native Mobile Application

There are two primary approaches through which the native mobile application can establish interactivity:

1. Web Sockets: In this approach, the mobile application leverages web sockets to connect to the web socket server, like the scenario outlined earlier. However, instead of a web application on the consumer side, it is now a native mobile application that establishes the web socket connection.

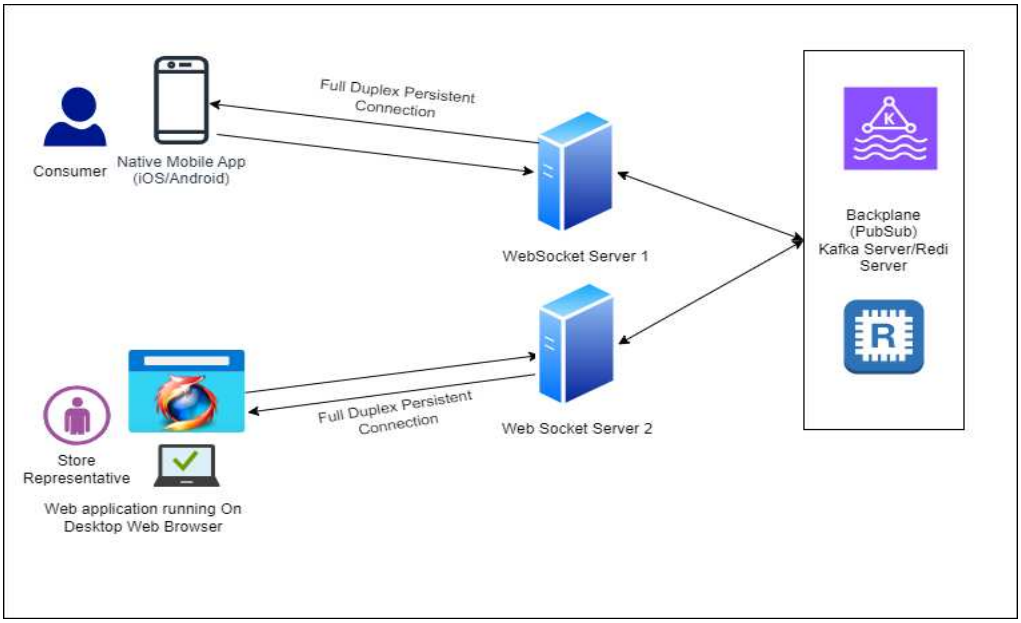

Figure. 3. Web and Mobile application using Web Sockets

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:06/Issue:01/Jauuary-2024 Impact Factor- 7.868 www.irjmets.com

2. REST-Based API Calls and Push Notifications: Alternatively, the mobile application can utilize standard REST-based API calls to push data from the loan applicant's side and receive updates from the other party through silent push notifications.

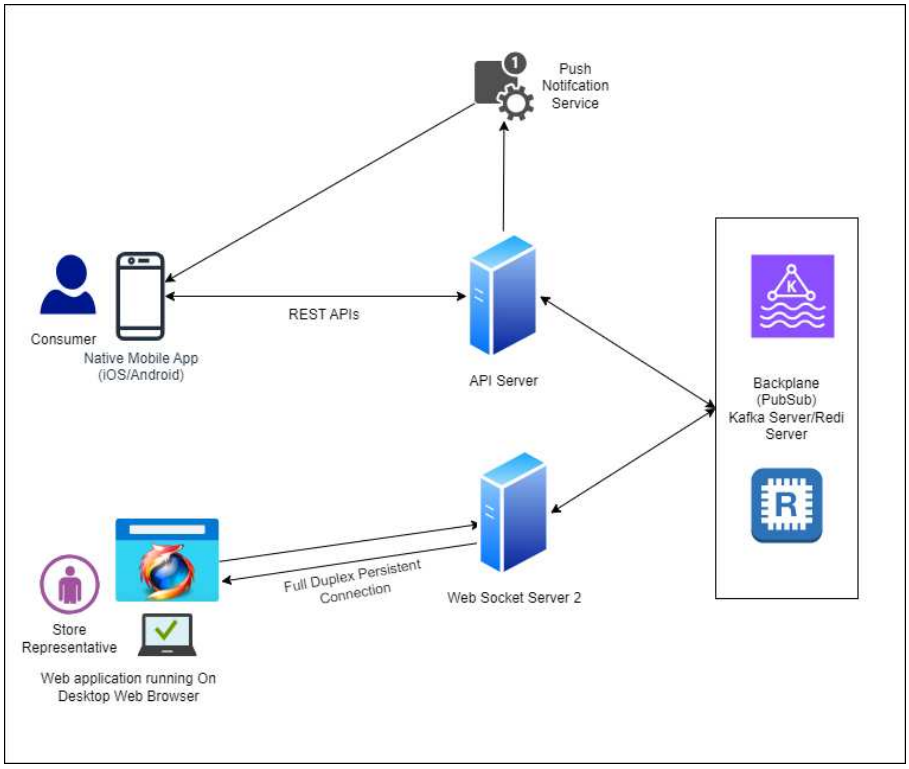

Figure. 4. Native mobile application using a combination of REST API and Push notifications.

There are several tools and frameworks commonly used to implement the mentioned approaches. Below, are some of the tools and frameworks associated with each approach:

- Mobile Application using web sockets:
  - WebSocket Libraries for Mobile Development:
    - socket.io-client-swift for iOS
    - Swift Web Socket for iOS
    - Ok Http for Android
- REST APIs can be built in any language of your choice.
- Web Socket server implementation we covered in Use Case 1.
- Push Notification Service: There are two approaches here. We can rely on platform-specific push notification services or use cloud-based services that abstract the connectivity for platform-specific needs.

1. AWS (Amazon Web Services):
   - Amazon Simple Notification Service (Amazon SNS): AWS offers Amazon SNS, a fully managed push notification service.
2. Azure (Microsoft Azure):
   - Azure Notification Hubs: Azure provides Azure Notification Hubs, a scalable and cross-platform push notification service.
3. Google Cloud Platform (GCP):
   - Firebase Cloud Messaging (FCM): Although Firebase is not strictly a part of GCP, it is closely integrated and widely used for push notifications. Firebase Cloud Messaging (FCM) is Google's cloud-based messaging platform that allows developers to send notifications to Android, and iOS.

- Silent notifications: Silent notifications are the same as any other push notification. The only difference is that these push notifications are not visible to the end user. These notifications would contain custom attributes which will help native mobile applications to identify that this is a silent push notification and should not be displayed to the end user.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:06/Issue:01/Jauuary-2024 Impact Factor- 7.868 www.irjmets.com

Interaction Between a Web Application and a Tablet Application

There are two primary approaches through which the native tablet application can establish interactivity, like that in Use Case 2. Both approaches would be identical to Use Case 2, with the only difference being that instead of a native mobile application, it would be a native tablet application for iPad.

Below, are some of the tools and frameworks associated with this approach:

- Tablet application (iPAD) using web sockets:
  - Swift Web Socket
  - Socket Rocket: Open-source Web Socket library for iOS, including iPad applications, developed by Square.

III. RECOMMENDATIONS AND BEST PRACTICES

Scaling

To ensure the scalability of Web Sockets based applications, the use of a backplane is essential. Redis, an open-source in-memory data store with publisher-subscriber features, is a widely adopted solution for implementing a backplane in Web Sockets based applications. Redis can effectively manage the distribution of messages across multiple server instances, allowing your application to handle a larger number of concurrent connections and messages.

Security

Implement a JWT (JSON Web Token) based authentication system for securing Web Socket endpoints. Web Sockets (WS) protocol has its secure version, called WSS, like HTTP and HTTPS.

Handling Web Socket Server Failures

It's essential to address server failures and provide the client application with the ability to reconnect seamlessly. Client libraries recommended above often come equipped with event handlers for different Web Socket connection events. Leveraging these event handlers can enhance the resilience of the application's connections. When a close event is received by the client application, use these handlers to trigger reconnection attempts, ensuring uninterrupted communication.

IV. CONCLUSION

In this research paper, we have explored the utilization of web sockets to enhance interactivity in hybrid applications, focusing on three distinct use cases: interactive sessions between two web applications, between a web application and a mobile device, and between a web application and a tablet device, all involving two parties. We have delved deep into each of these use cases, providing insights into the tools and frameworks employed to facilitate seamless interactivity. As the digital landscape continues to evolve and the demand for interactive user experiences remains pronounced, web sockets will continue to play a pivotal role in shaping the future of real-time communication in web and mobile applications.

V. REFERENCES

[1] Bhumij Gupta & Dr. M.P. Vani, An Overview of Web Sockets: The future of Real-Time Communication, International Research Journal of Engineering and Technology (IRJET), Volume 05, Issue 12, Dec 2018
[2] Varun Chopra, WebSocket Essentials – Building Apps with HTML5 WebSockets
[3] Baeldung, WebSockets with Spring, https://www.baeldung.com/websockets-spring
[4] Socket.IO, https://socket.io/
[5] SwiftWebSocket, https://github.com/tidwall/SwiftWebSocket
[6] Amazon Simple Notification Service (Amazon SNS), https://aws.amazon.com/what-is/push-notification-service/
[7] Azure Notifications Hub, https://learn.microsoft.com/en-us/azure/notification-hubs/notification-hubs-push-notification-overview